Jetson 카메라 응용 실습 결과보고서 (실측판)

2026-11-03 · 2팀

요약

확인 기준이 실측을 요구하는 항목은 팀이 교육장에서 실제 카메라(VIGI C540-W IP 카메라, Logitech C920)로 한 시험과 개인 원격 실측으로 판정하고, 실측으로 조건을 만들기 어려운 방식 비교·설계 검증은 가상 시험(합성 영상)으로 보충했다. 과제 1~6 확인 기준은 개방 어휘 검출(YOLOE 2/5)과 거리 측정 일부 항목을 빼고 충족했다. 재실 인원은 0~2명 구간에서 MAE 0.00~0.40명, 행동 패턴은 12회 중 11회, 거리는 카메라 1대를 8cm 옮겨 찍는 대체 방식으로 1.4m에서 −3.6%였고, 서비스 2개는 12시간 넘게 재시작 없이 돌았다.

과제	확인 기준	결과	충족
1 카메라 연동	(a)/(b) 2분 뒤 지연 비교·원인	VIGI (a) 96.2초 계속 증가 vs (b) 증가 없음, C920 (a) 0.85초 vs (b) 8ms	✅
	재연결 30초 이내	랜선 분리 후 두 서비스 재시작 없이 재연결(꽂은 시각 미기록), 송출 강제 종료 시험 4.6초 복구	✅
	glass-to-glass	C920 1초 이하, VIGI 4.7초(원격 뷰어 밀림 포함), 맥 카메라 RTSP 408ms	✅
2 물체 인식	FP16 추론 ⩽ 10ms	yolo11n 5.1ms(전체 11.48ms, 87fps), yolo11s 6.5ms	✅
	선 통과 10회 오차 ⩽ 1	10/10 (오차 0, 중복 0), MQTT 10건 모두 기록	✅
	벤치마크·INT8 mAP 하락	5행, INT8 3.2ms·mAP 0.486 (FP16 대비 −0.017)	✅
	조명×거리	0.93m·2m × 밝음·어두움 모두 검출률 95% 이상	✅
	개방 어휘 5종 중 4종 이상	yolo 1/5, 임베딩 재라벨 5/5, YOLOE 2/5(틀린 이름 0)	❌ (YOLOE)
3 재실 인원	MAE ⩽ 1.0, ±1명 ⩾ 80%	낮 30분 0.00·100%, 1명 칸 0.40·100%, 2명까지 10분 0.00·100%	✅
	빈 방 오판 ⩽ 10%	밤 657칸 0%	✅
	편향 원인 설명	30초 평활 창의 반응 지연(과소 편향), 붐비는 공간은 가림·크기(Mall)	✅

과제	확인 기준	결과	충족
4 행동 패턴	안정화 동작 10회 중 9회	만세 10/10, 중복 0	✅
	한팔 정지 오판 ≤ 1/10	약 12회 중 오판 1회	✅
	행동 12회 중 10회	11/12	✅
	혼동 사례 5건	5건	✅
5 거리 측정	실측 오차 1~2m ≤ 5%	의자 1.4m → 1.35m (−3.6%)	✅
	두 번째 지점·보정 25쌍·3~4m	꽃이 1.9m는 판정 불가, 나머지는 하지 못함	⚠️
6 응용 서비스	12시간 무중단	A3 12시간 37분, A1 13시간 7분, 재시작 0	✅
	카메라 분리·복귀	두 서비스 재시작 없이 자동 재연결	✅
	재부팅 3분 안 복귀	A1 약 1분, A3 2분 안 (개인 서비스 72초)	✅
	영상 저장 0건, 결과 형식 문서	저장 호출 없음, INTERFACE.md	✅

가장 크게 달라진 설계 변수는 **수신 방식 (a) → (b)**다. 같은 VIGI·같은 0.2초 처리에서 2분 뒤 지연이 96.2초에서 0으로 바뀌었고, 모델·해상도·하드웨어는 하나도 바꾸지 않았다. 다음은 추론 엔진(PyTorch → TensorRT FP16, 29.03 → 11.48ms, mAP −0.005)과 평활 창(순간 → 30초 median, 빈 방 오판은 0 그대로지만 사람이 들어온 뒤 약 15초 반응 지연)이다.

실습 환경

두 과제 모두 팀 공용 Jetson 1대에서 개인 계정과 개인 가상환경으로 진행했다.

항목	내용
보드	Jetson Orin Nano Super 8GB, JetPack 7.2.1, Ubuntu 24.04, CUDA 13.2, TensorRT 10.16
저장·전원·스왑	NVMe SSD 부팅, MAXN_SUPER, /swapfile 8GB
Python	~/eg2 가상환경, PyTorch 2.11 (jetson-ai-lab cu130), Ultralytics 8.4, sentence-transformers

항목	내용
카메라	VIGI C540-W (IP, Jetson 유선 포트에 직결, RTSP), Logitech C920 (USB). 원격 시험에는 맥 카메라를 RTSP로 송출한 가짜 IP 카메라도 사용
접속	Mac → Tailscale → SSH, 웹 화면은 브라우저로 포트 접속 (주소는 문서에 넣지 않음)
측정 도구	키트 폴더가 보드에 없어 직접 작성: `cam1.py`, `line_count.py` · `line2.py`, `light_dist.py`, `shift_stereo.py`, `bench2.py`, `occupancy_service.py`

지킨 규칙:

- 카메라 비밀번호는 Jetson의 설정 파일(권한 600)에만 두고 환경 변수로 읽는다. 코드·명령줄·저장소에 넣지 않는다.
- Tailscale 주소, 카메라 MAC 주소는 보고서·저장소에 넣지 않는다.
- 사진에 사람이 나오면 흐림 처리하고, 영상·모델 파일(.pt, .engine)은 올리지 않는다.
- 공용 보드라 재부팅이나 무거운 변환(TensorRT) 전에 팀원에게 알리고 `free -h`로 남은 메모리를 확인한다.

제출물 구성

제출물	파일	내용
기능 시험 기록	`runs/test_log.md`	T0~T4 항목마다 조건·결과·판정
결과 보고서	이 문서	공통 측정 조건, 과제 1~6 핵심 표, 원인과 조치, 설계 변수의 효과, 기준 대비 요약
확인 문제 답안	이 문서 확인 문제 1~6	1-1~6-5, 문항당 2~5줄, 실측 근거 포함
응용 서비스	Jetson에서 가동 + `*.service`	systemd로 부팅 시 자동 시작
결과 형식 문서	별도 문서	MQTT 토픽·필드·단위·예시·갱신 주기·조회 주소
상세 기록	별도 파일	재시험, 실패, 수정 과정

실측과 가상 시험의 구분

과제 기준이 '실측'을 요구하는 항목은 모두 실제 카메라로 재고, 실측으로는 조건을 만들기 어렵거나 설계를 미리 검증하는 항목만 가상 시험(정답을 아는 합성 영상에 실제 YOLO를 돌려 채점)으로 했다. 가상 시험 결과는 모두 '보조 시험 — 가상'으로 표시했다.

구분	항목	이유
실측	과제 1 지연·재연결, 과제 2 벤치마크·선 통과·물품, 과제 3 교육장 MAE·빈 방, 과제 4 동작·행동, 과제 5 실측 오차표, 과제 6 12시간·분리·재부팅, 심화 디코딩·모델 공유·자기 감시	확인 기준이 실제 카메라·현장 값을 요구함
가상 (보조)	과제 2 조명×거리 추세, 과제 3 출입 vs 직접 동시 운영, 과제 5 세 방식 비교·가상 영상 세 지점·알람·TTC, 심화 혼합 보정·기준선 6/12cm·거리 자기 감시	정답 인원·위치·속도를 프레임마다 알아야 채점되거나, 같은 조건을 반복해 설계 변수만 바꿔 보는 항목

가상 시험은 렌즈 흐림·압축·두 카메라 시간 차가 없어 실측보다 좋게 나온다. 그래서 판정은 실측으로 하고, 가상 시험은 방식 간 비교와 경향 확인에만 썼다.

과제 1. 카메라 연동과 기준선

최신 1장 리더(b)를 쓰면 두 입력 모두 지연이 쌓이지 않았고, 그대로 읽기(a)는 RTSP에서 2분 뒤 약 96초까지 밀렸다. 재연결은 4.6초 만에 자동 복구되어 기준(30초)을 만족했다.

입력	해상도	FPS	열기 / 첫 프레임 (ms)	프로세스 CPU	(a) 2분 뒤 지연	(b) 2분 뒤 지연	glass-to-glass	재연결
RTSP (맥 카메라)	1280x720	32.3	1442 / 1468	18.8%	약 96,000ms (계속 증가)	일정 (쌓이지 않음)	408ms (386 / 414 / 425)	4.6초
USB (C920)	1280x720	15.1	257 / 647	11.4%	833ms (일정)	31ms (일정)	교육장 시험	교육장 시험

- 조건: 처리 지연 0.2초, 각 120초 측정. RTSP는 2026-11-02 17:34–17:38, USB는 22:57–23:01(원격, 교육장 조명 꺼짐)에 쟀다.
- (a)의 차이: 처리(0.2초)가 카메라보다 느리면 못 읽은 프레임이 버퍼에 쌓인다. RTSP는 버퍼가 커서 끝없이 쌓였고, USB는 드라이버 버퍼가 약 4장이라 4 × 0.2초 ≈ 0.8초에서 멈췄다.
- (b)는 별도 스레드가 계속 읽고 최신 1장만 남겨 지연이 쌓이지 않는다. 대신 읽기 스레드 때문에 CPU가 늘었다(USB 7.0% → 20.1%).
- USB는 압축·네트워크 단계가 없어 첫 프레임이 RTSP보다 약 820ms 빨랐다. FPS 15.1은 어두운 환경에서 C920이 노출을 늘리려고 스스로 낮춘 값이다.

- 재연결 시험: 맥 송출을 강제로 꺼서(22:48:11) 끊겼고, 프로세스 재시작 없이 22:48:16에 복구했다.
- glass-to-glass: 젯슨 웹 화면의 밀리초 시계를 거울로 맥 카메라에 비추고, 화면 캡처에서 '위 시계 − 영상 속 시계'를 3회 계산했다.
- USB glass-to-glass와 재연결(랜선 분리)은 팀이 교육장에서 VIGI·C920으로 따로 시험했다(아래 팀 실측).

교육장 실측 (팀, VIGI·C920)

교육장의 실제 카메라에서도 같은 결론이 나왔다. (a)는 VIGI에서 2분 뒤 96.2초까지 밀렸고 (b)는 지연이 늘지 않았다.

입력	실측 FPS	해상도	첫 프레임	프로세스 CPU	(a) 2분 뒤	(b) 2분 뒤	glass-to-glass
C920 (USB)	30.0	1280x720	638ms	35.7%	약 0.85초에서 멈춤	약 8ms	1초 이하 (640x480 뷰어)
VIGI (RTSP)	27.1	848x480	1,216ms	6.3%	96.2초, 계속 증가	증가 없음	새로고침 직후 4.7초, 61초 뒤 6.6초

조건 — 프레임마다 0.2초 처리 지연, 120초 측정. VIGI glass-to-glass는 영상에 찍힌 카메라 시계(NTP, Jetson과 1초 안으로 확인)와 브라우저 시계를 비교했다.

- 계산과 맞는다: VIGI는 초당 25장이 들어오는데 5장만 처리하므로 매초 0.8초씩 밀린다. 120초 × 0.8 = 96초로 실측과 같다. C920은 드라이버 버퍼가 차면 더 쌓이지 않아 0.85초에서 멈춘다.
- (b)의 대가: 모든 프레임을 계속 받아야 해서 C920의 CPU가 7.4% → 44.3%로 올랐다.
- VIGI glass-to-glass는 켜 둘수록 늘고 새로고침하면 줄었다. 늘어난 부분은 카메라가 아니라 원격 뷰어(MJPEG) 전송이 밀린 것이다.
- 재연결: VIGI 랜선을 뽑았다 꽂아도 두 서비스 모두 재시작 없이 다시 연결했다(각 1회). 재실 서비스(A1)는 5.7초 안에 끊김을 표시하고, 다시 연결된 뒤 2초 안에 정상으로 돌아왔다. 다만 랜선을 다시 꽂은 시각을 적지 않아 '꽂은 뒤 30초 안'은 숫자로 보이지 못했고, 이 부분은 위 송출 강제 종료 시험(4.6초)으로 확인했다.
- 설치 위치: 사람이 화면에 들어오기 시작하는 거리는 C920 1.08m, VIGI 0.86m다. 1m 근처에서는 두 카메라 모두 머리가 잘려 동작 시험은 2~3m에서 했다.

확인 문제 1

1-1. `appsink drop=true max-buffers=1 sync=false`

- `max-buffers=1` : appsink 대기열에 프레임을 1장만 보관한다.
- `drop=true` : 대기열이 차면 기다리지 않고 옛 프레임을 버리고 새 프레임을 넣는다.
- `sync=false` : 타임스탬프에 맞춰 기다리지 않고 도착 즉시 넘긴다.
- 세 옵션을 합치면 리더 (b)처럼 항상 최신 1장만 받는다.

1-2. RTSP를 UDP에서 TCP로 바꾸면

- 좋아지는 점: 패킷 손실이 없어 화면 깨짐이 없고, 방화벽·NAT·ssh 터널을 통과한다. 이번 구성(ssh -R)도 TCP라서 가능했다.
- 나빠지는 점: 손실 시 재전송을 기다리는 동안 뒤 프레임도 멈춰 지연이 튀고, 네트워크가 나쁘면 버퍼가 쌓인다.

1-3. GOP를 FPS와 같게(1초) 두는 이유

- 디코더는 I-프레임부터만 화면을 만든다. 처음 접속하거나 재연결하면 다음 I-프레임까지 기다려야 하는데, GOP가 1초면 이 대기가 최대 1초다.
- GOP를 길게 하면 대역폭은 줄지만 첫 화면과 복구가 그만큼 늦다. 이번 송출도 `-g 30`(30fps에서 1초)이었다.

1-4. 보드 CPU 부하가 가장 작은 카메라

- CSI 카메라. 센서 영상이 MIPI로 젯슨의 VI·ISP 하드웨어에 바로 들어와 압축 풀기·USB·네트워크 처리가 없다.
- USB는 USB 처리와 MJPEG 풀기, IP 카메라는 네트워크와 H.264 풀기가 CPU에 든다. 측정에서도 프로세스 CPU가 USB 11.4% < RTSP 18.8%였고, IP 카메라 디코딩을 하드웨어(NVDEC)로 넘기면 CPU가 크게 줄었다(팀 4.2% 대 30.1%).

1-5. 리더 (b)에서도 e2e 지연이 300ms일 때 코드가 줄일 수 없는 부분

- 카메라 안: 센서 노출 시간(30fps면 약 33ms)과 카메라 내부 처리·압축.
- 전송: 네트워크 이동 시간.
- 표시: 모니터 주사율(60Hz면 최대 약 17ms)과 브라우저 그리기.
- 코드가 줄일 수 있는 것은 대기열과 추론 시간뿐이고, 대기열은 (b)로 이미 없앰다.

과제 2. 물체 인식

교육장 실측 (팀, VIGI 서브 스트림)

FP16 엔진은 정확도를 0.005만 잃고 2.5배 빨랐고(추론 5.1ms), 속도와 정확도를 함께 보면 yolo11s FP16이 가장 균형이 좋았다.

모델	한 장 전체 (ms)	추론만 (ms)	p95 (ms)	FPS	mAP50-95	보드 전력
PyTorch yolo11n	29.03	—	30.47	34.4	0.508	13.2W
PyTorch yolo11s	29.97	—	33.3	33.4	0.576	—
TensorRT FP16 yolo11n	11.48	5.1	14.87	87.1	0.503	10.9W
TensorRT FP16 yolo11s	12.95	6.5	13.63	77.2	0.576	—
TensorRT INT8 yolo11n	9.62	3.2	10.27	104.0	0.486	—

측정 조건 — 입력 640, 1280x720 영상 200회, mAP는 coco128. 위 세 줄은 서비스 2개가 같은 GPU를 쓰는 상태, yolo11s FP16·INT8은 서비스를 잠시 끄고 변환·측정(조건이 더 가벼움). 변환 시간 yolo11s FP16 404초, INT8 550초(coco128 128장 보정).

- PyTorch에서 yolo11s가 yolo11n과 거의 같은 속도인 것은, 시간의 대부분이 모델 계산보다 PyTorch의 GPU 실행 부담이기 때문이다. 엔진으로 바꾸면서 줄어든 약 17.5ms가 그 부분이다.
- INT8은 가장 빠르지만 mAP가 FP16보다 0.017(약 3.4%) 낮다. yolo11s FP16은 PyTorch yolo11n보다 빠르고(12.95 vs 29.03ms) 정확하다(0.576 vs 0.508).
- 변환 중 남은 메모리가 최소 456MB까지 떨어졌다. 서비스를 끄고 변환하면 1.6~1.9GB가 남아 안전했다.

선 통과 (VIGI 서브, yolo11n, 2~3m): ByteTrack으로 10회(IN 5 / OUT 5)를 가로질러 10회 모두 맞게 셈다(오차 0, 중복 0). IN/OUT마다 MQTT `aiot/vision/test/event`로 메시지가 나갔고 구독 기록에 10개가 모두 남았다(필드 ts, dir, id, in, out). 앞서 기본 추적기(BoT-SORT)로 한 9회도 9/9였다.

조명 × 거리 (VIGI 서브, 칸마다 30초)

조명	0.93m	2m
밝음	검출률 100% · 신뢰도 0.949	95.1% · 0.884
어두움 (낮, 앞쪽 조명만 끔)	100% · 0.938	100% · 0.927

- 거리는 방에서 설 수 있는 0.93m와 2m로 했다. 낮에 앞쪽 조명만 끈 상태라 창문 빛과 카메라 자동 노출로 화면 밝기가 117~125로 유지돼 검출이 거의 떨어지지 않았다. 더 어둡고 먼 조건은 아래 보조 시험(가상)으로 추세를 봤다.

물품 인식 (VIGI 서브, 약 1m, 라벨 목록 9개)

물건	COCO에 있음	yolo	relabel	yoloe
우산 (접힘)	있음	bed	우산	못 잡음
컵	있음	remote, cup	컵	cup
꽂이 (흰 구멍 판)	없음	cell phone	꽂이	못 잡음
상자	없음	suitcase	상자	cardboard box
줄자	없음	cell phone	줄자	못 잡음
정답		1/5	5/5	2/5 (틀린 이름 0)

- yolo는 모르는 물건에 비슷한 COCO 이름을 붙였고, 같은 물건도 시험마다 이름이 달랐다(상자: suitcase, umbrella, refrigerator).
- relabel은 모두 맞혔지만, 정답이 모두 든 9개짜리 목록이라 유리한 조건이다.

- yoloe는 잡은 물건에는 틀린 이름을 붙이지 않았다. 우산·꽂이·줄자를 못 잡은 것은 "pen holder" 문구가 실제 모양(흰 구멍 판)과 달랐고 줄자는 손에 쥐면 작게 보이기 때문이다.

TensorRT 벤치마크 (개인 측정)

TensorRT FP16 엔진의 추론 시간은 5.09ms로 기준(10ms 이하)을 만족했고, 전체 처리 60.6fps로 카메라(30fps)를 충분히 따라간다. INT8은 FP16보다 8%만 빠르고 mAP50-95가 0.0245 떨어져 기본값으로 쓰지 않는다.

모델	추론 평균 (ms)	추론 p95 (ms)	전체 평균 (ms)	FPS	mAP50-95	mAP50	전력 (W)	파일 (MB)
PyTorch yolo11n (FP32)	25.14	26.09	32.69	30.6	0.5076	0.6749	12.41	5.6
TensorRT yolo11n FP16	5.09	6.88	16.50	60.6	0.5063	0.6757	11.60	8.5
TensorRT yolo11n INT8	4.66	6.66	16.70	59.9	0.4818	0.6540	10.80	5.8
TensorRT yolo11s FP16 (큰 모델)	8.87	12.67	21.06	47.5	0.5765	0.7460	13.21	22.0

측정 조건 — 입력 640, batch 1, bus.jpg 300회 반복(예열 30회 제외), mAP는 coco128(128장), 전력은 tegrastats VDD_IN 평균. INT8 보정은 coco128로 85초. 과제 6 서비스가 같은 GPU에서 동시 가동 중, 보드 RAM 사용 약 6.5~6.7GB. 2026-11-02, `bench2.py`*, 총 1,718초.*

- PyTorch → FP16: 추론은 약 5배(25.14 → 5.09ms) 빨라졌지만 전체는 2배(32.69 → 16.50ms)다. 남은 약 11ms는 CPU에서 하는 전처리(크기 조절)와 후처리(NMS)다. mAP는 거의 같다(0.5076 → 0.5063).
- INT8: FP16 대비 추론 0.43ms(8%) 빠르고 전력 0.8W 적지만, mAP50-95가 0.5063 → 0.4818(약 4.8%) 떨어졌다. 작은 모델이라 속도 이득이 작아 정확도 손해를 감수할 이유가 없다.
- 큰 모델(yolo11s) FP16: mAP50-95가 0.0702 오르고 추론 8.87ms로 여전히 10ms 이하다. 먼 사람을 봐야 하는 재실 인원처럼 정확도가 중요하면 이쪽이 낫다.
- 팀 측정과의 차이: 개인 측정은 과제 6 서비스가 같은 GPU에서 동시에 돌던 중이라 INT8(4.66ms)·yolo11s(8.87ms)가 팀 측정(3.2ms·6.5ms)보다 느렸다. mAP는 yolo11s 0.5765 대 0.576으로 같다.
- 한계: 프로세스 메모리는 네 모델을 한 프로세스에서 차례로 재서 최대값(3,595MB)만 남아 모델별로 구분되지 않는다.

추적 + 가상 선 통과 (개인 측정)

선 양쪽에 여유 폭(화면 너비 6%)을 두자 실제 IN 5 / OUT 5를 정확히 셈다(오차 0, 기준 ≤1). 여유 폭이 없을 때는 같은 시험에서 IN 7 / OUT 6으로 3번 더 셈다.

판정 방식	실제 IN / OUT	센 IN / OUT	오차
박스 하단 중앙 + 외적 부호 (여유 폭 없음)	5 / 5	7 / 6	3
+ 선 양쪽 여유 폭 6% (히스테리시스)	5 / 5	5 / 5	0

측정 조건 — 맥 카메라 RTSP 1280x720, TensorRT yolo11n FP16 + ByteTrack, 화면 가운데 세로선 (위→아래로 그음, 왼→오 = IN), 약 42fps. 통과할 때마다 MQTT `aiot/space/lab1/line` (QoS 1)으로 발행하고 `runs/line2_log.jsonl` 에 기록. 2026-11-03, `line2.py`.

- 원인: 카메라에 가까워 박스가 화면을 꽉 채우고 판정점(박스 하단 중앙)이 화면 아래 끝에 붙어 있었다. 선 근처에서 박스가 조금만 흔들려도 판정점이 선을 넘었다 돌아와 같은 사람을 여러 번 센다(확인 문제 2-4의 경우).
- 수정: 여유 폭 안에서는 판정을 보류하고, 반대쪽 여유 폭을 완전히 넘었을 때만 1번 센다. 코드는 `--margin` 옵션 하나로 바꿨다.

COCO에 없는 물품 5종 — 세 방식 비교 (개인 측정)

임베딩 재라벨(②)이 4/5로 가장 정확했고, 개방 어휘 YOLOE(③)는 프롬프트·임계값을 바꿔도 최고 3/5로 기준(4/5 이상)에 못 미쳤다. YOLO 라벨 그대로(①)는 0/5로, 모르는 물품에 비슷한 COCO 라벨을 붙였다. 팀 시험에서는 임베딩 재라벨이 5/5로 기준을 넘었지만, YOLOE는 팀 2/5·개인 최고 3/5로 두 시험 모두 기준에 못 미쳤다.

물품 (프롬프트)	① YOLO 그대로	② 임베딩 재라벨 (유사도)	③ YOLOE 1차 (임계값 0.15)	③ YOLOE 2차 (0.05 + 프롬프트 수정)
빗 (hair brush)	toothbrush / tv	O (0.659)	O 10/10	—
약통 (pill bottle)	bottle	O (0.676)	O 14/14	—
보조배터리 (power bank → portable battery pack)	cell phone / remote	O (0.641 / 0.631)	없음 (1/9)	파우치 (오인식)
파우치 (cosmetic pouch → small zipper bag)	frisbee	O (0.560 / 0.561)	없음	O 10/10
충전기 (phone charger → white usb wall charger)	없음 / couch	X (없음 / 파우치)	없음	보조배터리 (오인식)
정답	0/5	4/5	2/5	최고 조합 3/5

측정 조건 — 맥 카메라 RTSP 1280x720, 물품을 화면 가운데 가까이 들고 8초 동안 매 프레임 판정, 가장 많이 나온 답을 기록(9~121프레임). ① TensorRT yolo11n FP16(conf 0.25, 사람 제외, 크고 가운데인 박스), ② ①의 박스를 EmbeddingGemma 2로 임베딩해 5개 텍스트와 비교(유사도 0.20 미만이면 YOLO 라벨 유지), ③ yoloe-11s-seg에 5개 클래스를 텍스트로 지정. 2026-11-03, `vocab2.py`*.*

- ①의 오인식은 모양이 비슷한 COCO 클래스로 간다: 약통→bottle, 보조배터리→cell phone·remote, 빗→toothbrush, 둥글고 납작한 파우치→frisbee.
- ②는 YOLO가 박스만 잡으면 잘 맞힌다. 하지만 YOLO가 못 잡은 충전기는 재라벨할 박스가 없어 실패했고, 엉뚱한 박스(couch)를 잡으면 그 박스를 재라벨해 틀렸다. 유사도는 0.56~0.68로 임계값 0.20보다 훨씬 높아, 후보에 없는 물체도 걸러내지 못할 것이다.
- ③는 모양이 뚜렷한 물품(빗, 약통)은 확실히 맞히지만, 네모난 작은 상자류(보조배터리, 충전기)는 헷갈렸다. 임계값을 0.05로 낮추자 '없음'이 줄었지만, 대신 다른 후보 이름을 자신 있게 붙였다. 프롬프트 문구를 바꾸자 파우치는 없음 → 정답으로 바뀌었다.
- 안경을 처음 후보로 넣었을 때는 측정자가 쓴 안경을 대신 잡아 빗을 '안경'으로 96/97 오인식했다. 후보 목록에 현장에 항상 보이는 물체가 있으면 오인식이 생긴다.
- 다음 시도: 물품마다 현장 사진을 모아 YOLOE의 시각 프롬프트(예시 이미지)를 쓰거나, ②와 ③를 합쳐 ③가 잡은 박스를 ②로 재확인한다.

보조 시험 — 조명 × 거리별 사람 검출 (가상, 합성 영상)

교육장 실측(`light_dist.py`)과 별도로, 밝기와 거리를 더 넓게 바꿔 추세를 보려고 합성 영상으로 돌렸다. 밝기 30% 이상이면 12m까지 88% 이상 검출했고, 밝기 5%에서는 8m 이상에서 거의 못 찾았다(0~2%). 어두움이 거리보다 훨씬 큰 변수다.

거리 (사람 키 px)	밝음 100%	어둑 30%	어두움 10%	아주 어두움 5%
2m (818px)	100% · 0.84	100% · 0.85	100% · 0.83	50% · 0.44
5m (327px)	100% · 0.82	100% · 0.78	68% · 0.61	35% · 0.46
8m (204px)	100% · 0.74	100% · 0.71	73% · 0.55	2% · 0.21
12m (136px)	98% · 0.63	88% · 0.57	55% · 0.42	0% · 0.11

칸 = 검출률 · 평균 신뢰도. 가상 시험(합성 영상) — 서 있는 사람 사진을 1280x720 화면에 거리별 크기(키 1.7m)로 배치하고 밝기를 낮추며 어두울수록 큰 센서 노이즈를 넣음. 조건마다 20장, yolo11n FP16, 검출 = IoU≥0.5 & conf≥0.4. 2026-11-03, `light2.py`*.*

- 신뢰도는 멀수록(0.84 → 0.63), 어두울수록 떨어진다. 어두운 화면은 노이즈가 윤곽을 덮어 먼 사람부터 사라진다.
- 현장 적용: 야간·소등 후 교실은 IR 카메라나 보조조명이 필요하다.
- 한계: 사진을 키우고 줄인 합성이라 실제 원근·압축·조명 방향은 반영하지 못한다. 판정은 교육장 실측표로 한다.

확인 문제 2

2-1. `.pt` → `.engine` 변환이 빨라지는 이유 세 가지, 다른 보드에 복사하면 안 되는 이유

- 층 합치기: Conv·BN·활성화 같은 연속 연산을 하나의 커널로 묶어 GPU 호출과 메모리 왕복을 줄인다.
- 정밀도 낮추기: FP32를 FP16/INT8로 바꿔 계산량·메모리를 줄이고 Tensor Core를 쓴다.
- 커널 자동 선택: 빌드 때 이 GPU에서 계산 방식을 실제로 재 보고 가장 빠른 것을 고르며, 입력 크기를 고정해 메모리 배치도 미리 정한다.
- 복사 금지: 엔진은 만든 GPU 구조와 TensorRT·CUDA 버전에 맞춰 고른 커널이 들어 있다. 다른 보드나 버전에서는 열리지 않거나 느려서, 그 보드에서 다시 빌드해야 한다.

2-2. 평균보다 p95 지연을 함께 보는 이유

- 상시 서비스는 매 프레임을 제시간에 처리해야 한다. 평균은 가끔 생기는 느린 프레임을 가린다.
- 느려지는 원인: 다른 프로세스와 GPU 공유(이번에도 과제 6 서비스가 동시에 가동), 발열로 인한 클럭 저하, 메모리 정리.
- 10fps에서 5%는 1분에 30장이다. 그 순간에 프레임이 밀리거나 알람이 늦어지므로 꼬리 지연을 보고 여유를 정해야 한다.

2-3. 박스 하단 중앙을 쓰는 이유, 코드 없이 IN·OUT을 바꾸는 법

- 하단 중앙은 사람의 발, 즉 바닥에서 서 있는 위치다. 바닥에 그은 선과 같은 평면에서 비교된다.
- 박스 중심은 키·자세(앉기, 팔 뻗기)·가림·원근에 따라 위치가 흔들려, 실제로 선을 넘지 않아도 넘은 것처럼 보일 수 있다.
- 방향 바꾸기: 선의 두 끝점 순서를 바꿔 아래→위로 긋는다. 외적 부호가 반대가 되어 왼→오 이동이 OUT이 된다. `line2.py`에서는 `--line 0.5,0.9,0.5,0.1`로 바꾼다.

2-4. 선 위에서 멈추다가 되돌아간 경우

- 발이 선을 넘었다 돌아오면 IN 1, OUT 1이 된다. 실제로 들어왔다 나간 것과 같고 순인원은 0이라 버그는 아니다.
- 다만 선 위에서 박스가 흔들리면 IN·OUT이 여러 번 쌓일 수 있다. 선 양쪽에 여유 폭(히스테리시스)을 두거나 여러 프레임 연속으로 넘어야 인정하면 줄어든다.

2-5. `relabel` 모드에서 유사도 임계값 0.20을 없애면

- 유사도는 항상 후보 중 가장 높은 것을 고르므로, 후보에 없는 물체도 가장 비슷한 라벨을 강제로 받는다.
- 그러면 모르는 물체에 틀린 라벨을 자신 있게 붙이는 문제가 오히려 늘어난다. 임계값은 근거가 약하면 원래 YOLO 라벨을 유지하는 안전장치다.

과제 3. 재실 인원 카운터

교육장 실측 (팀, 재실 서비스 A1)

A1은 VIGI 화면 전체에서 yolo11n(입력 640, conf 0.4)으로 사람을 직접 세고, 30초 median으로 평활해 10초마다 MQTT(retain)로 낸다. 0~2명 구간에서 세 기준(MAE ⩽ 1.0, ±1명 ⩾ 80%, 빈 방 오판 ⩽ 10%)을 모두 만족했다.

평가	칸 수	MAE (명)	±1명 이내	빈 방 오판
밤 빈 방 (11/2 19:00 ~ 11/3 06:00)	657	—	—	0%
낮 30분 (11:24~11:53, 정답 모두 0명)	30	0.00	100%	0%
1명 칸 (시험 기록과 대조)	5	0.40	100%	—
2명까지 10분 (2명 1·1명 2·0명 7칸)	10	0.00	100%	0%

- 정답 인원은 예측값을 보기 전에 정했다.
- 편향 원인: 1명 칸 5개 중 2개에서 순간 값은 1, 30초 중간값은 0이었다. 둘 다 사람이 들어온 지 10~15초 된 시점으로, 창의 절반이 차기 전이었다. 이 서비스의 오차는 가림이나 크기가 아니라 평활 창이 만드는 지연(과소 편향)에서 나왔다.
- 밤 기록 1,105개에서도 순간 값과 중간값이 다른 10개는 모두 사람이 드나들던 시간이었고, 빈 방에서는 순간 검출부터 0이었다.
- 한계: 평가가 0~2명 구간뿐이라, 사람이 많고 겹치는 경우는 아래 Mall Dataset 시험으로 보완했다.
- MQTT 이벤트는 메모리에 최근 20개만 남는다. 시험 전에 `mosquitto_sub`로 파일 기록을 켜고 한 줄이 남는지 먼저 확인했다.

붐비는 공간 (개인, Mall Dataset)

YOLO는 붐비는 실내에서 사람을 항상 덜 셈고(서비스 기본값 MAE 16.2명, 평균 31명 중), 입력 크기·임계값을 바꾸고 직선 보정식을 더하자 시험 구간 MAE가 3.1명까지 줄었다.

조건	MAE (명)	편향 (명)	±2명 이내	속도 (ms/장)	시험 MAE 보정 전 → 후	보정 후 ±2명 이내
yolo11n FP16 · 640 · conf 0.4 (서비스 기본값)	16.20	−16.20	0.0%	21.9	16.81 → 3.90	41.4%
yolo11n FP16 · 640 · conf 0.25	12.78	−12.78	0.0%	20.2	13.20 → 3.20	47.6%
yolo11s FP16 · 640 · conf 0.4	17.22	−17.22	0.0%	24.3	17.80 → 4.62	33.5%
yolo11s FP16 · 640 · conf 0.25	13.75	−13.75	0.1%	25.5	14.11 → 3.93	38.3%
yolo11s PyTorch · 960 · conf 0.25	10.35	−10.35	0.9%	82.2	10.70 → 3.09	48.7%

측정 조건 — Mall Dataset(CUHK) 2,000장, 640x480 실내 고정 카메라, 프레임마다 사람 수 정답(13~53명, 평균 31.2명). 사람(class 0)만 센다. 편향 = 센 값 − 정답. 보정식(정답 ≈ a × 센 값 + b)은 앞 1,000장으로 만들고 뒤 1,000장에서만 채점했다(예: 서비스 기본값 a = 1.54, b = 7.5). 2026-11-03, `occ_eval.py`, `occ_calib.py`.

- 왜 덜 세나: 위에서 내려다보는 각도에 먼 사람은 몇십 픽셀로 작고 서로 겹친다. 정답은 머리가 보이는 사람 전부인데 YOLO는 몸이 충분히 보여야 박스를 그린다. 편향이 MAE와 같아 흔들림이 아니라 꾸준한 누락이다.
- 가장 큰 설계 변수는 입력 크기다. 640 → 960으로 키우자 MAE 13.75 → 10.35로 줄었고, 대신 3배 이상 느려졌다(1초에 1번 세는 서비스에는 충분). conf를 0.4 → 0.25로 낮추는 것도 효과가 있었다(16.20 → 12.78). 모델만 키우는 것(n → s)은 효과가 없었다.
- 보정식은 꾸준한 누락을 거의 없앤다(편향 −1명 이내). 단, 카메라 각도·방 구조가 바뀌면 a, b가 달라지므로 방마다 정답을 조금 세어 다시 구해야 한다.
- 교육장(1~2명)에서는 MAE 0.40명 이하로 잘 맞았지만, 붐비는 공간에서는 입력 960·conf 0.25·방별 보정식이 필요하다.

보조 시험 — 출입 계수 vs 직접 계수 동시 운영 (가상, 합성 영상)

두 방식을 같은 영상에서 동시에 돌리려면 프레임마다 정답 인원을 알아야 해서 합성 영상으로 했다. 직접 계수는 MAE 0.14명으로 정답을 거의 따랐고, 출입 계수는 놓친 IN·OUT이 쌓여 끝났을 때 1명 틀렸다.

방식	MAE (명)	편향 (명)	±1명 이내	끝났을 때 (정답 4)
직접 계수 (방 안 발 위치, 중앙값 5프레임)	0.14	−0.13	99%	4
출입 계수 (문 선 IN−OUT 누적)	0.63	−0.29	99%	5

가상 시험(합성 영상) — 카메라 1대(1280x720, 높이 2.5m, 25° 아래), 사람 6명이 차례로 들어오고(두 명은 붙어서, 한 명은 앞사람 뒤에 가려짐) 2명이 나감. yolo11n + ByteTrack, conf 0.3, 문 선 여유 폭 0.25m, 56초. 2026-11-03, `vcount.py`.

- 출입 계수는 IN 5/6, OUT 0/2를 셈다. 붙어 들어오면 한 명을 놓쳤고, 나간 두 명의 OUT을 모두 놓쳤다. 출입 계수의 오류는 사라지지 않고 누적된다.
- 선택: 방 안이 한 화면에 보이면 직접 계수를 기본으로 하고, 출입 계수는 보조로 쓰되 직접 계수로 주기적으로 바로잡는다(심화 혼합 보정 참고).

과제 4. 행동 패턴 인식

교육장 실측 (팀, 안내판 서비스 A3)

A3(`pose_app.py`)는 yolo11n-pose로 17관절을 구하고, 동작은 0.6초 다수결로 안정화한 규칙으로, 행동은 정면 점수·이동 속도·체류 시간 규칙으로 판정한다(VIGI 서브, 2~3m). 네 기준을 모두 만족했다.

시험	기준	결과	판정
안정화 동작 10회 (만세)	9회 이상, 깜빡임 ≤ 1	10/10, 중복 0	✅
한팔 들고 정지	손흔들기 오판 ≤ 1/10	약 12회 중 11회 정답, 오판 1회	✅
행동 패턴 12회	10회 이상	11/12 (머무름 재측정 반영)	✅
혼동 사례	5건 이상	5건	✅
악수 (2명)	—	원래 기준 0회 → 완화 뒤 약 3/4	참고

행동 12회 혼동 행렬 (1차) — 행 = 정답, 열 = 판정

정답 \ 판정	지나감	힐끗 봄	관심 있음	머무름
지나감	3	0	0	0
힐끗 봄	0	3	0	0
관심 있음	1	0	2	0
머무름	0	0	3	0

- 머무름 3회가 모두 관심 있음으로 나온 것은 시험자가 정면을 봤기 때문이다(연기 오류). 옆을 보고 다시 한 3회는 모두 머무름이었다. 같은 자리에 서 있어도 시선 방향 하나로 판정이 갈렸다.
- 한팔 정지의 오판 1회와 만세 사이 손흔들기는, 팔을 빠르게 올리고 내릴 때 손목이 팔꿈치 위를 좌우로 지나가 흔들기 규칙(1.5초 안 방향 전환 2번)에 걸린 것이다. 회차가 약 2.3초로 짧아 0.6초 다수결이 걸러 내지 못했다. 손목이 팔꿈치 위에 0.5초 이상 있을 때만 흔들기로 보면 줄어들 것이다(미적용).
- 악수는 원래 기준(손목 거리 어깨 너비의 0.5배, 손목 신뢰도 0.5)에서 한 번도 잡히지 않았다. 맞잡은 손목은 서로 가려 신뢰도가 떨어지고, 옆모습에서는 어깨 너비가 좁아져 거리 기준이 거의 0이 된다. 1.0배·0.3으로 풀어 약 4회 중 3회를 잡았고, 가까이 서기만 했을 때는 한 번도 나오지 않았다.

raw vs 안정화 (만세 10회, 같은 VIGI 영상을 두 모드가 동시에 받음)

항목	raw	안정화 (0.6초 다수결)
만세 인식	10/10	10/10
같은 회차 중복	0	1
회차 사이 다른 라벨	한팔들기 12번	손흔들기 4번
판정 시각	기준	평균 약 0.3초 늦음 (0.2~0.5초)

조건 — 2026-11-03 18:57~18:58, 한 사람이 약 2초 간격으로 만세 10회. 두 앱의 MQTT 기록에서 0.2초 안에 겹친 같은 라벨은 같은 이벤트로 셈다.

- raw는 두 팔이 동시에 올라가지 않는 순간마다 한팔들기가 끼어 12번 깜빡였다. 안정화는 이 깜빡임을 모두 걸렀고, 대가로 판정이 약 0.3초 늦었다(0.6초 창의 60%가 차는 시간과 맞음).
- 안정화 쪽의 손흔들기 4번은 다수결 문제가 아니라 손흔들기 규칙이 안정화 모드에만 있어서 생긴 것으로, 위 한팔 시험과 같은 원인이다.
- 설계 변수: 즉시 반응이 필요한 알람은 raw, 기록·통계는 안정화를 쓴다.

개인 시험 (embedding_gemma2 9단계 결과 재사용)

YOLO 포즈 키포인트로 동작(손흔들기·박수·팔짱·가만히)과 행동(관심 등)을 규칙과 학습(GRU) 두 방식으로 판정했다. 학습 모델이 '가만히(idle)'를 내면 규칙의 손흔들기를 덮어쓰던 문제를 고쳤다.

항목	내용
프로그램	`pose_app.py` (실시간 인식 + 웹 대시보드), `train_pose.py` (GRU 학습)
동작 모델	`models/action.pt` — 라벨 arms_crossed, clap, idle, wave, 창 30프레임
행동 모델	`models/behavior.pt`
혼동 사례 1	학습 모델이 idle로 판단하면 규칙 기반 손흔들기를 덮어써 손흔들기가 잡히지 않았다 → idle일 때는 규칙 결과를 유지하도록 수정
혼동 사례 2	영어 라벨(interested 등)이 한글 순위표와 맞지 않아 관심 비율이 0%로 나왔다 → 라벨을 한글로 변환
혼동 사례 3	손을 연속으로 흔들면 1회로 기록(동작이 바뀔 때만 기록) → 따로 세려면 약 2초 손을 내려야 한다

raw vs 안정화(smooth) — 동작마다 3회씩, 센 횟수:

동작 (실제 3회)	raw	smooth	비고
만세	3	3	raw는 올리고 내릴 때 한팔들기로 6번 깜빡임, smooth는 깜빡임 없음
한팔들기	3	3	둘 다 정확
손흔들기	0	3	raw는 모두 한팔들기로 인식, smooth는 1~1.5초 뒤 손흔들기로 바뀜(판정 지연)

동작 (실제 3회)	raw	smooth	비고
악수	5~6	5	맞잡고 가만히 있으면 손목이 가려져 미검출, 두 사람이 겹치기만 해도 악수로 오인식
측면 만세	6	5	옆모습에서 반대쪽 어깨가 가려져 과다 카운트

행동 패턴(규칙, 개인 시험): 지나감 3/3, 힐끗 봄 1/3, 관심 있음 3/3, 머무름 0/3 → 12회 중 7회. 규칙이 시선이 아니라 머리 방향을 봐서, 걸으며 고개를 돌리면 정면 시간이 짧게 잡혔고(힐끗 봄→지나감), 옆을 보며 서 있으면 한쪽 어깨가 가려져 판정이 갱신되지 않았다(머무름→힐끗 봄). 교육장 팀 시험(11/12)과의 차이는 카메라 위치·조명과 시험자 동작 차이로 보인다.

학습 모델(GRU): 동작 모델 최고 검증 정확도 0.976(팔짱 30개 중 3개를 손흔들기로 오판), 행동 모델 1.000(지나감 검증 조각 2개라 참고 수준). 녹화에 참여하지 않은 사람에게 15번 중 13번(87%) 맞혔고, 약점은 빠른 박수(1/3, 팔짱으로 오판)였다. 규칙으로 불가능한 박수·팔짱을 학습이 인식했다.

측정 조건 — 9단계(embedding_gemma2 실습) 결과 재사용. USB 웹캠, yolo11n-pose, smooth = 0.6초 다수결 + 1.5초 손목 움직임, GRU window 30. 2026-10-26.

과제 5. 거리 측정

같은 모델 웹캠 2대와 키트가 없어, C920 1대를 8cm 옮겨 두 번 찍는 대체 방식(`shift_stereo.py`)으로 정지한 물체만 쟀다. 의자 등받이(1.4m)는 −3.6%로 기준을 만족했지만, 두 번째 지점은 판정하지 못했다.

물체	실제 (줄자)	측정	오차	판정
의자 등받이	1.4m	1.35m	−3.6%	✅ (≤ 5%)
꽃이 윗면	1.9m (앞면 기준)	2.12m	+12% 이하	판정 불가

조건 — 초점거리는 embedding_gemma2 과제에서 같은 C920으로 만든 보정 파일(fx ≈ 681px, 640x480, 재투영 RMS 0.169px) 사용. 도구의 가상 시차 시험은 3점 모두 0.02px 안.

- 왜 두 번째 점은 판정 불가인가: 꽃이는 윗면을 재었는데 줄자는 앞면까지 재서 기준점이 달랐다. 오차가 최대 +12%라는 것만 말할 수 있다.
- 카메라를 손으로 옮기는 방식의 한계: 모니터에 단 채 옮긴 처음 두 번은 카메라가 크게(12~16°) 돌아갔다. 책상 위에서 다시 했어도 약 2° 돌고 기울어, 거리를 아는 두 점(상자 1.7m, 스크린 2.0m)으로 회전과 기울기를 빼고 나머지로 검증했다.
- 하지 못한 것: 스테레오 보정 25쌍·재투영 오차, 3~4m 측정, 측정 불가 조건 4가지, 사람별 거리. 두 카메라를 고정한 진짜 스테레오가 있어야 하는 항목이라, 방식 비교는 아래 보조 시험(가상)으로 대신했다.

- 이론 비교: 확인 문제 5-1 식($\Delta Z \approx Z^2 \cdot \Delta d/(f \cdot B)$)으로 보면 기준선 8cm는 거리 제곱에 비례해 오차가 커져 3m 이상에서는 기준(8%)을 넘기 쉽다.

보조 시험 — 세 방식 비교와 알람 (가상, 합성 영상)

대체 방식(카메라 1대 이동)으로는 움직이는 사람과 다리 가림을 잴 수 없어, 가상 카메라 2대로 방식을 비교했다. 어깨 삼각측량(stereo)은 평균 오차 0.09~0.10m, 발목 방식(ankle1)은 0.55~0.66m였고 다리가 가려지자 위치를 못 냈다(0/30).

시나리오	ankle1 (발목 → 바닥)	stereo (어깨 삼각측량)	fused (stereo + 한쪽 추정)
트인 곳 걷기 (8m → 1.5m)	38/40 · 0.66m	40/40 · 0.10m	40/40 · 0.10m
책상 뒤 걷기 (다리 가림)	0/30 · —	30/30 · 0.09m	30/30 · 0.09m
옆으로 비켜 감 (한 카메라에만)	24/32 · 0.55m	23/32 · 0.09m	23/32 · 0.09m

칸 = 위치가 나온 프레임 · 평균 오차. 가상 시험(합성 영상) — 가상 카메라 2대(1280x720, 초점거리 960px, 높이 1.2m, 간격 1.0m)가 본 합성 화면에 yolo11n-pose를 실제로 돌림. 2026-11-03, `vstereo.py`.

가상 영상 세 지점 거리: 1.5 / 2.5 / 4.0m에서 측정 1.502 / 2.505 / 3.999m(오차 0.1 / 0.2 / 0.0%). 계산식 자체는 맞다는 확인이고, 실제 오차는 위 실측(1.4m −3.6%)으로 판정한다.

접근 알람과 충돌 예상 시간(TTC): 고정 타겟 알람은 정답과 0.1초 안에서 울렸다(접근 2.6초, 이탈 7.7초, 헛알람 없음). 다가오는 사람의 속도별 결과:

실제 속도 (m/s)	추정 속도 (m/s)	충돌 위험 알람 때 거리 (m)	근접 알람 때 거리 (m)	충돌 위험이 먼저 울린 시간 (s)
0.5	0.49	0.82	1.47	−1.2 (늦음)
1.0	1.00	1.94	1.47	0.5
1.5	1.49	2.89	1.39	1.0

가상 시험(합성 영상) — 카메라 2대, 사람 2명, 속도 = 최근 1초 위치의 직선 기울기, 충돌 위험 = 접근 속도 > 0.5m/s 이고 TTC < 2초. 2026-11-03, `vprox.py`.

- 빠를수록 충돌 위험 알람이 더 멀리서, 더 일찍 울린다. 천천히 다가오는 대상(0.5m/s)은 근접 알람이 막아야 한다.
- 가상 시험은 두 카메라 시간 차가 0이라, 실제 웹캠(약 80ms)에서는 1m/s에 약 8cm 위치 오차가 더 생긴다.

과제 6. 응용 서비스

교육장에는 CO_2·온습도 센서는 있지만 사람을 보는 수단이 없다. 그래서 방의 인원을 숫자로 내는 재실 서비스(A1)와, 안내판 앞 사람이 지나가는지 멈춰 보는지를 이벤트로 내는 안내판 서비스(A3)를 만들었다. 영상은 보드 밖으로 내보내지 않고, 다른 팀은 MQTT와 REST로 가져다 쓴다.

확인 기준	결과 (팀)	판정
12시간 무중단	A3 12시간 37분(758행, 평균 21.1fps, 재연결 0, 재시작 0), A1 13시간 7분(재연결 0, 저장 이미지 0)	✅
카메라 분리·복귀	두 서비스 재시작 없이 자동 재연결	✅
재부팅 3분 안 복귀	A1 약 1분, A3 2분 안 (11/3 16:29)	✅
결과 형식 문서	INTERFACE.md	✅
운영 모드 영상 저장 0건	두 서비스 저장 호출 없음 (뷰어는 운영 때 끔)	✅

장애 시험에서 찾아 고친 것

- A1: 랜선 분리 시험에서 52초 동안 오래된 값을 정상으로 보였다. 마지막 프레임의 나이로 판단해 5.7초 안에 끊김을 표시하게 고쳤다.
- A3: 프로세스는 살아 있는데 영상만 멈췄다(재연결 조건이 '실패 50회 누적'). 2.5초 시간 기준 재연결과, 120초 동안 프레임이 없으면 스스로 종료해 systemd가 다시 띄우는 감시를 넣었다.

자원 계획 (계산 vs 실측, 팀)

항목	계산	실측
두 서비스 메모리	약 1.2GB + 1.3GB ≈ 2.5GB (보드의 33%)	남은 메모리 1.3~2.2GB, 12시간 중 최소 422MB
카메라 4대 디코딩	CPU 4 × 30.1% ≈ 코어 1.2개, NVDEC ≈ 0.17개	1대 30.1% / 4.2%
GPU	A1 추론 약 41ms, A3 약 21fps	평균 25.7%, 최대 99%
전력	9.1W × 24시간 ≈ 218Wh/일	평균 9.1W, 최대 17.3W

측정 — jtop 대신 tegrastats(1분 간격 774회).

- 카메라를 늘리면 메모리가 먼저 바닥난다. 두 서비스를 한 프로세스로 합쳐 PyTorch·CUDA 실행 환경을 한 번만 올리거나 TensorRT 엔진으로 바꾸는 것이 먼저다(심화에서 합치기 실측: 프로세스 메모리 3,356 → 1,693MB).
- USB 설정을 서비스 시작 전에 복구하는 처리는 두 서비스가 IP 카메라를 써서 넣지 않았다(방법은 확인 문제 6-2).

- 재부팅하면 서비스가 아닌 프로세스(기록기 등)는 사라진다. 포트는 서비스가 8001을 쓰면 시험 도구는 8002 이후로 열었다.

아래는 같은 방식으로 개인이 따로 띄운 재실 서비스의 가동 기록이다.

재실 인원 서비스 v1

항목	내용
입력	교육장 VIGI IP 카메라 (stream2), 최신 프레임만 유지, 끊기면 2초마다 재연결
추론	YOLO yolo11n, 1초마다 사람(class 0) 수 계산, 30초 창 중앙값으로 평활
발행	10초마다 MQTT `aiot/space/lab1/occupancy` (retain, QoS 1), 상태 `aiot/service/occupancy/status`
REST	`:8095/occupancy`, `:8095/health` (CPU·메모리·GPU, 저장 이미지 0장)
기록	1분마다 `logs/service_날짜.jsonl`, 이벤트 `logs/events.jsonl`, `tegrastats.log`
자동 복구	`Restart=always`, 비밀번호는 EnvironmentFile(chmod 600)에 보관
개인정보	영상·이미지는 저장하지 않고 인원 숫자만 보드 밖으로 내보냄

12시간 가동 기록

11-02 18:01부터 12시간 동안 서비스는 한 번도 재시작하지 않았고, 카메라는 718분 중 717분 정상, 재연결 0회, MQTT는 내내 연결돼 있었다.

항목	값
기간	2026-11-02 18:01 ~ 11-03 06:01 (12시간)
기록	1분마다 718행 / 기대 720행, 최대 공백 62초
서비스 재시작	0회 (가동 시간 초기화 0회, 시작 이벤트 1회)
카메라 lab1	정상 717/718행, 재연결 0회
MQTT 연결	718/718행

항목	값
추론 시간	평균 42.7ms / 최대 50.3ms
보드 CPU	평균 25.5% / 최대 91.7%
보드 메모리	평균 4,163MB / 최대 6,988MB
보드 GPU	평균 68.7% / 최대 99.7%
전력 (VDD_IN)	평균 9.1W / 최대 18.3W
재실 인원	평균 0.0명 / 최대 0.0명 (야간 빈 교육장)

측정 조건 — VIGI IP 카메라 stream2, yolo11n PyTorch 입력 640, conf 0.4, 1초에 1번 세기, 30초 중앙값, 10초마다 발행. CPU·메모리·GPU·전력은 보드 전체 값이다. 같은 밤 23:00~01:35에 같은 보드에서 벤치마크 등 다른 실험을 함께 돌려 최대값과 평균 GPU가 그만큼 높게 나왔다. `occ_summary.py`.

- systemd의 NRestarts는 15로 나오지만, 이는 18:01 이전 포트 8090 충돌로 시작에 실패하던 재시도까지 센 값이다. 12시간 안에는 가동 시간이 한 번도 초기화되지 않았다.
- 빈 방 오판: 야간 718번 발행 모두 0명(0%, 기준 10% 이하).

자원 계획 (계산치 vs 실측치)

자원	계산	계산치	실측치
GPU (이 서비스)	카메라 1대 × 1회/초 × 추론 42.7ms	약 4.3%	보드 전체 평균 68.7% (다른 실험 포함)
GPU (FP16 엔진으로 바꾸면)	1 × 1 × 5.1ms (과제 2 벤치마크)	약 0.5%	—
세 프로그램 동시 (재실·통행·거리)	1×10 + 15×5 + 15×5 ms (확인 문제 6-1)	약 16%	심화 실측 GPU 평균 57~60% (PyTorch)
카메라 늘리기	10대 × 1회/초 × 42.7ms	약 43%	—
전력	—	—	평균 9.1W

- 추론 시간 42.7ms는 벤치마크의 PyTorch 25ms보다 길다. 같은 GPU를 다른 실험과 나눠 썼기 때문이다. FP16 엔진으로 바꾸면 카메라를 여러 대 늘려도 GPU 여유가 충분하다.
- 보드 메모리는 최대 6,988MB(전체 약 7.5GB)까지 올랐다. 카메라를 늘리거나 LLM을 함께 올리면 GPU보다 메모리가 먼저 바닥난다.

재부팅 복구 시험

보드를 재부팅하자 72초 만에 서비스가 자동으로 다시 떠서 REST에 응답했고, 카메라도 정상으로 인원을 세고 있었다(기준 3분 이내).

항목	값
재부팅 명령	2026-11-03 08:52:06 (`sudo reboot`)
복귀 확인	`:8095/health` 첫 응답까지 72초
복귀 직후 값	lab1 count 0, camera 정상, age_s 0.0, reconnects 0 (08:53:10 발행)

측정 — 2초마다 `curl` 로 `:8095/health` 를 확인해 첫 응답 시각을 쟀다. 자동 시작은 systemd `WantedBy=multi-user.target`, 카메라 비밀번호는 EnvironmentFile에서 읽는다.

카메라 1대 분리 시험

카메라 하나를 끊어도 나머지 카메라는 계속 정상으로 셈고, 끊긴 카메라는 송출이 돌아오자 프로세스 재시작 없이 자동으로 다시 연결됐다.

시각	사건	lab1 (VIGI)	home (맥 카메라 RTSP)
01:28:25	시험 서비스 시작 (카메라 2대)	정상	정상
01:30:48	home 송출·터널 강제 종료 (카메라 분리)	정상 (계속 셈)	끊김(재연결 중)
01:33:03	home 송출 다시 시작	정상	복구 (끊긴 지 134.3초)
01:33:03~01:33:15	송출이 안정되기 전 끊김·복구 5번 반복 (각 1.9초 만에 복구)	정상	이후 정상, 재연결 6회

시험 조건 — 12시간 서비스는 그대로 두고 같은 코드로 시험용 서비스를 따로 띄움(포트 8098, 토픽 `test/space`). 분리는 맥의 RTSP 송출과 ssh 터널을 강제 종료해 대신했다. 기록 `logs/events.jsonl` . 2026-11-03.

- 134.3초는 송출을 다시 켜기까지 기다린 시간이고, 서비스는 송출이 돌아온 뒤 2초 안에 다시 받기 시작했다.
- 복구 직후 12초 동안 끊김·복구가 반복돼 재연결 횟수가 6으로 부풀려졌다. 개선: 일정 시간(예: 5초) 안정적으로 프레임이 들어와야 '복구'로 기록하고, 재연결 간격을 점점 늘린다.

심화(B) 항목 (실측)

과제	심화 항목	결과
1	1080p 하드웨어(NVDEC) vs CPU 디코딩	개인: 1080p 30fps CPU 94%(GStreamer CPU)·65%(OpenCV) → 12%, 최대 180fps / 팀: VIGI 2560x1440 25fps CPU 30.1% → 4.2%
4	GRU 학습, 녹화에 참여하지 않은 사람으로 시험	15번 중 13번(87%), 규칙으로 불가능한 박수·팔짱 인식
6	세 프로그램을 하나로 합쳐 모델 공유	프로세스 메모리(RSS) 합 3,356 → 1,693MB
6	서비스 자기 감시	사람이 센 값과 비교해 오차 ≥2명이면 '보정 필요' 알림, 야간 빈 방 자동 점검

과제 1 심화 — 1080p 디코딩: 하드웨어 vs CPU

방식	흘리는 속도	처리 FPS	프로세스 CPU (코어 1개 = 100%)	걸린 시간 (s)
GStreamer nvv4l2decoder (하드웨어, NVDEC)	최대 속도	180	64%	5.0
GStreamer nvv4l2decoder (하드웨어, NVDEC)	실시간 30fps	30	12%	30.2
GStreamer avdec_h264 (CPU)	최대 속도	98	226%	9.2
GStreamer avdec_h264 (CPU)	실시간 30fps	30	94%	30.2
OpenCV VideoCapture (FFmpeg, CPU)	최대 속도	157	266%	5.7
OpenCV VideoCapture (FFmpeg, CPU)	실시간 30fps	30	65%	30.0

측정 조건 — 1920x1080 30fps H.264 High 30초(900프레임, GOP 30, B-프레임 없음), 젯슨 MAXN_SUPER. 처리 FPS = 900 ÷ 걸린 시간. GStreamer는 BGRx까지 변환 후 버림. CPU 사용률 = 프로세스 (user+sys) CPU 시간 ÷ 걸린 시간. 2026-11-03, `dec_compare.py`.

- CPU 디코딩은 1080p 1대에 코어 0.65~0.94개를 쓴다. 6코어 보드에서 6~9대면 디코딩만으로 CPU가 찬다.

- GStreamer CPU가 OpenCV보다 무거운 것은 색 변환(videoconvert)까지 CPU가 하기 때문으로 보인다. 하드웨어 방식은 색 변환도 nvvidconv(VIC)가 한다.
- 팀은 실제 VIGI 메인 스트림(2560x1440)을 GStreamer로 60초씩 받아, NVDEC와 CPU(openh264dec) 모두 25fps에서 CPU 4.2% 대 30.1%를 쟀다. 서브 스트림은 화소가 약 9분의 1이고 CPU도 6.2% 대 38.0%의 약 6분의 1이라, 두 서비스는 서브 스트림을 쓴다. 조건은 다르지만 하드웨어 디코딩이 CPU를 5~8배 줄인다는 결론은 같다.
- 한계: 지금 앱이 쓰는 pip 판 OpenCV에는 GStreamer가 없어 하드웨어 디코딩을 바로 못 쓴다.

과제 6 심화 — 세 프로그램 합치기 (모델 공유)

방식	프로세스 RSS 합 (MB)	GPU 사용률 평균	재실 (1회/초)	통행 (15회/초)	거리 (15회/초)
따로 (프로세스 3개)	3,356	57%	1.0회/초 · p50 45ms	15.0회/초 · p50 38ms	8.9회/초 · p50 112ms
합침 (프로세스 1개)	1,693	60%	1.0회/초 · p50 54ms	15.0회/초 · p50 39ms	7.5회/초 · p50 133ms

측정 조건 — yolo11n PyTorch(CUDA), 입력 bus.jpg 1280x720 고정, 재실 960·통행/거리 640, 거리는 640x360 SGBM 추가, 워밍업 20초 뒤 60초. 합침은 통행·거리가 640 모델 하나를 번갈아 씀. 2026-11-03, `merge_bench.py`*.*

- 합치면 PyTorch·CUDA 런타임을 한 번만 올려 프로세스 메모리가 절반으로 줄었다. 대신 모델을 번갈아 써서 거리 작업 지연이 112 → 133ms로 늘었다.
- 거리(15회/초)는 두 방식 모두 목표에 못 미쳤다. CPU의 SGBM(약 70ms)이 병목이다.
- 한계: RSS는 공유 라이브러리를 프로세스마다 다시 세서 '따로'가 실제보다 크게 나온다. 보드 전체 RAM은 측정 중 다른 팀원 프로그램이 바뀌어 비교에서 뺐다.

과제 6 심화 — 서비스 자기 감시

서비스 코드는 그대로 두고 옆에서 도는 감시 프로그램(`occ_monitor.py` , 웹 :8097)을 만들었다. 상태가 바뀔 때마다 MQTT `aiot/service/occupancy/monitor` (retain)로 알린다.

점검	방법	알림
사람이 센 값과 비교	하루 2회(10:00, 15:00) 사람이 센 인원을 넣으면 서비스 값(최근 30초 중앙값)과 비교	오차 ≥2명 한 번 또는 ≥1명 두 번 연속 → 보정 필요, 1시간 안에 입력 없으면 → 점검 누락
사람 없이 자동 점검	새벽 02:00~05:00에는 빈 방이어야 함	1명 이상이 10분 넘게 → 야간 오검출 의심

점검	방법	알림
서비스·카메라 상태	10초마다 REST 조회	카메라 이상 / 서비스 이상

- 기능 시험(알림 동작 확인용으로 값을 일부러 넣음): 서비스 0~1명일 때 6·7명을 넣자 오차 −5~−6으로 '보정 필요'가 떴고, 이어 오차 −1이 나오자 '두 번 연속' 규칙으로 알림이 유지됐다. 처음 켤 때 카메라 상태 한글 표기('정상')를 이상으로 본 버그와, 감시 시작 전 점검 시각을 누락으로 친 버그를 고쳤다. 2026-11-03.

보조 시험(가상) — 과제 3 심화: 혼합 보정

압축한 하루(10분, 30명 방문) 동안 출입 계수는 놓친 IN·OUT이 쌓여 끝에 5명이 남았고, 30초마다 직접 계수로 바로잡는 혼합 방식은 드리프트를 0으로 없앰다.

방식	MAE (명)	편향 (명)	±1명 이내	끝났을 때 오차 (드리프트)
① 직접 계수	1.11	−1.07	66%	0
② 출입 계수	1.73	+1.69	56%	+5
③ 혼합 (30초마다 보정)	0.81	−0.72	79%	0

가상 시험(합성 영상) — 30명이 무작위 시각에 들어와 40~300초 머물다 나감(20%는 붙어서 들어옴), 보정 조건 = 직접 계수가 3초 동안 같은 값 + 문 근처에 아무도 없음. 2026-11-03, `vhybrid.py`.

- 약점: 사람이 많을 때(13~14명) 직접 계수가 가려진 사람을 덜 세서, 그때 보정하면 혼합도 따라 틀린다(369초 13→10, 정답 12). 개선안: 인원이 적을 때만 보정하거나 한 번에 ±1명까지만 바꾼다.

보조 시험(가상) — 과제 5 심화: 기준선 6cm vs 12cm

기준선을 2배로 늘리자 거리 오차가 평균 약 2.2배 줄어 $\Delta Z \propto 1/B$가 대체로 성립했다. 12cm는 모든 거리에서 기준을 통과했지만 6cm는 2m에서 5%를 넘었다.

거리 Z (m)	6cm 평균 오차 · 오차율	12cm 평균 오차 · 오차율	오차 비 (6cm ÷ 12cm)
1	0.041m · 4.1%	0.025m · 2.5%	1.7
2	0.109m · 5.4%	0.045m · 2.3%	2.4
3	0.240m · 8.0%	0.113m · 3.8%	2.1
4	0.177m · 4.4%	0.182m · 4.5%	1.0
5	0.422m · 8.4%	0.106m · 2.1%	4.0

가상 시험(합성 영상) — 평행 카메라 2대(초점거리 960px, 높이 1.2m), 화면마다 독립 노이즈 σ=6, yolo11n-pose 어깨 중점 시차로 Z = f·B/d, 거리마다 12번. 2026-11-03, `vbaseline.py`.

- 측정 오차는 실제로 잰 시차 흔들림(1~3px)을 넣은 이론식 $\Delta Z = Z^2 \cdot \Delta d/(f \cdot B)$과 거의 같았다. 실측한 대체 방식(기준선 8cm)의 1.4m −3.6%도 이 범위 안이다.
- 4m·5m는 12번뿐이라 흔들렸다. 실측 비교는 하지 못했다.

보조 시험(가상) — 과제 6 심화: 거리 자기 감시

고정 기준 물체(벽)의 거리를 계속 재다가 카메라가 틀어지면 '보정 필요'를 낸다. 0.2° 틀어지자 5프레임 만에 알림이 울렸고 틀어지기 전 헛알람은 0회였다.

오른쪽 카메라 회전	0°	0.05°	0.1°	0.2°	0.3°	0.5°
측정 벽 거리 (정답 3.0m)	3.022m	3.042m	3.114m	3.291m	3.478m	3.840m
기준값 대비	+0.7%	+1.4%	+3.8%	+9.7%	+15.9%	+28.0%

가상 시험(합성 영상) — 기준선 0.12m, 무늬 있는 벽 3.0m, StereoSGBM, 규칙 = 기준 대비 3% 넘는 프레임이 5번 연속. 2026-11-03, `vwall.py`.

- 0.2° 회전은 화면을 약 3.4px 밀어 시차 38.4 → 35.0px, Z ≈ 3.29m로 계산과 측정이 같다. 실제 11/3 재시험에서도 카메라를 떼었다 다시 달자 재투영 오차가 6~7px → 76.7px로 튀어, 이런 감시가 필요한 이유를 보여 준다.
- 한계: 합성 영상이라 평소 흔들림이 거의 0이다. 현장에서 하루 흔들림을 재고 기준(3%)을 다시 정해야 한다.

하지 않은 심화

Windows 웹캠 RTSP 송출(스크립트 `win/start_win.ps1`만 작성), YOLO 파인튜닝, DeepStream, BRIO 4K 비교, 재실 인원을 넣은 CO_2 예측, 기준선 6/12cm의 실측 비교(가상 시험만 함).

확인 문제 3

3-1. 1920px 폭 영상, 맨 뒷자리 사람 키 70px

- 입력 640, 조각 없음: 70 × 640 ÷ 1920 ≈ 23px → 안정 기준 32px에 못 미친다.
- imgsz 960: 70 × 960 ÷ 1920 = 35px, 또는 입력 640 + 조각 2x2: 70 × 640 ÷ 960 ≈ 47px. 둘 중 하나면 32px을 넘는다(둘 다 쓰면 70px).
- 과제 3 시험에서도 입력 640 → 960이 MAE를 13.75 → 10.35명으로 줄였다.

3-2. 편향 −0.4

- 편향 = 센 값 − 정답이므로 음수는 덜 센다는 뜻이다.

- 먼저 할 조치: ① conf를 낮춘다(0.4 → 0.25, 과제 3에서 MAE 16.2 → 12.8), ② 입력 크기를 키우거나 조각 검출로 작은 사람을 살린다. 그래도 남는 꾸준한 누락은 직선 보정식으로 바로잡는다.

3-3. `--stat median`과 `p75`

- median은 창 안 값의 가운데, p75는 아래에서 75% 지점 값이라 더 큰 쪽을 따른다.
- 책상·파티션에 가려지면 사람을 놓칠 뿐 없는 사람을 더 세지는 않는다. 그래서 창 안의 낮은 값은 가림 때문일 가능성이 커서, 높은 쪽(p75)이 실제에 가깝다.

3-4. 출입 계수에서 하루 끝 재실 −3

- 원인 ①: 나갈 때 선 위에서 흔들리거나 추적 ID가 바뀌어 OUT을 두 번 셈다(과제 2에서 여유 폭 없이 IN 7/OUT 6이 나온 것과 같다).
- 원인 ②: 여러 명이 겹쳐 들어오거나 다른 문으로 들어와 IN을 놓쳤다.
- 대책: 선 양쪽 여유 폭(히스테리시스), 모든 출입구에 카메라, 방이 비는 시간(새벽)에 0으로 초기화하고 음수는 0으로 자른다. 직접 계수로 주기적으로 누적 값을 바로잡는다.

3-5. MQTT `retain`을 켜지 않으면

- 재시작한 에이전트는 다음 발행(우리 서비스는 최대 10초)까지 현재 인원을 모른다. 그동안 값이 없거나 0으로 오해해 제어를 잘못할 수 있다. retain을 켜면 브로커가 마지막 값을 접속 즉시 보내 준다.

3-6. 조각 검출(2x2)에서 경계에 걸친 사람

- 조각별 박스를 전체 화면 좌표로 옮긴 뒤 한 번 더 NMS(같은 사람으로 겹치는 박스 중 하나만 남기기)를 한다.
- 그래도 겹침 0.15를 두는 이유: 겹침이 없으면 경계의 사람이 반쪽씩 잘려 어느 조각에서도 제대로 검출되지 않는다. 겹치게 자르면 적어도 한 조각에는 온전하게 들어간다.

확인 문제 4

4-1. 특징 벡터 53차원

- 관절 17개의 (x, y) 좌표 34 + 관절별 신뢰도 17 + 정면 응시 점수 1 + 이동 속도 1 = 53.
- 신뢰도를 넣는 이유: 가려지거나 화면 밖인 관절의 좌표는 엉터리다. 신뢰도가 있어야 모델이 어느 좌표를 믿을지 배운다.

4-2. 손 흔들기 규칙의 오판·놓침

- 오판: 팔을 든 채 칠판을 지우거나 머리를 긁을 때도 손목이 좌우로 반복돼 손흔들기로 판정된다.
- 놓침: 옆모습이나 뒷모습이면 손목의 x좌표 변화가 작고, 어깨 아래에서 손만 작게 흔들거나 1.5초보다 느리게 흔들면 못 잡는다. 연속으로 흔들면 1회로만 기록됐다.

4-3. 나란히 선 두 사람의 악수 오판 줄이기

- 두 사람의 몸통이 서로 마주 보는지(어깨선 방향·코 방향)를 조건에 더한다.

- 맞닿은 손목이 두 사람 골반 중심 사이에 있고, 0.5초 이상 유지되며 위아래로 작게 흔들리는지도 본다. 팀 교육장 시험에서는 가까이 서기만 했을 때 오검출 0번이었다.

4-4. `--window 30 --stride 5` 와 검증 분할

- 학습 샘플 하나 = 연속 30프레임(30fps면 약 1초), 다음 샘플은 5프레임 뒤에서 시작한다.
- 이웃 샘플은 30프레임 중 25프레임이 같다. 무작위로 섞으면 거의 같은 샘플이 학습과 검증에 나눠 들어가 정확도가 부풀려진다. 그래서 시퀀스 뒷부분 20%를 따로 떼어 검증한다.

4-5. '관심 있음' 300건 vs 실제 멈춘 사람 50명

- 가장 의심되는 원인은 추적 ID가 끊기면서 같은 사람이 여러 번 새 사람으로 세인 것이다. 사람(ID) 단위로 중복을 없애고, 잠깐 사라진 ID는 일정 시간 안에 다시 잇는다.

확인 문제 5

5-1. f = 1000px, B = 0.08m, d = 40px

- $Z = f \cdot B \div d = 1000 \times 0.08 \div 40 = 2.0$m.
- $\Delta Z \approx Z^2 \cdot \Delta d \div (f \cdot B) = 4 \times 0.5 \div 80 = 0.025$m = 약 2.5cm.
- 우리 대체 방식도 기준선 8cm였다.

5-2. 거리 4m

- 시차 $d = 80 \div 4 = 20$px, $\Delta Z = 16 \times 0.5 \div 80 = 0.1$m = 약 10cm.
- 1m(d = 80px, ΔZ ≈ 0.6cm)와 비교하면 거리 4배에 오차는 16배(거리의 제곱)다.

5-3. `--num-disp 64` , scale 0.5일 때 최소 거리

- 최대 시차 = 64 ÷ 0.5 = 128px, 최소 거리 = 80 ÷ 128 ≈ 0.63m.
- 0.5m를 재려면 시차 160px이 필요하다 → num-disp를 80 이상(16의 배수인 96)으로 올리거나 기준선을 줄인다.

5-4. `--board 9x6`

- 체커보드 안쪽 꼭짓점(검은 칸 4개가 만나는 점)의 가로×세로 개수다. 10x7칸 보드의 안쪽 꼭짓점은 9x6이다.
- `--board 10x7` 로 넣으면 그런 배치가 없어 체커보드를 한 장도 찾지 못한다.

5-5. 재투영 0.3px인데 기준선이 80mm → 96mm

- 칸 크기(`--square`)를 잘못 넣었을 가능성이 가장 크다. 재투영 오차는 모양만 보므로 축척이 1.2배 틀려도 작게 나온다. 실제 칸을 자로 다시 재서 넣는다. 우리는 체커보드 칸 크기를 삼각측량으로 다시 재서(24.1 대 24.0mm) 축척을 확인했다.

5-6. 무늬 없는 흰 벽이 검게(측정 불가) 나오는 이유

- SGBM은 왼쪽 영상의 작은 조각과 가장 비슷한 조각을 오른쪽 영상의 같은 행에서 찾는다. 흰 벽은 어느 조각이나 똑같아 비용이 평평하고, 유일성 검사를 통과하지 못해 측정 불가로 나온다.

확인 문제 6

6-1. GPU 점유율과 LLM

- 재실 1회 × 10ms + 통행 15회 × 5ms + 거리 15회 × 5ms = 1초에 160ms → 약 16%(SGBM 20ms는 CPU).
- LLM이 질의마다 GPU를 2초씩 쓰면 그동안 비전 추론이 밀려 프레임을 건너뛰거나 지연이 튀고, 통행 선 통과를 놓칠 수 있다.
- 완화: 최신 1장 리더로 밀린 프레임을 버리고, 작은·양자화된 LLM을 쓰며, 질의를 큐에 넣어 한 번에 하나만 처리하고 답 길이를 제한한다.

6-2. 재부팅 때 초기화되는 USB 웹캠 설정

- systemd의 `ExecStartPre=` 지시어로 서비스 시작 전에 설정을 다시 넣는다.
- 예: `ExecStartPre=/usr/bin/v4l2-ctl -d /dev/video0 -c focus_automatic_continuous=0 -c focus_absolute=0 -c power_line_frequency=2` (자동 초점 끄기, 초점 고정, 60Hz 깜빡임 방지).

6-3. 두 프로그램이 모두 `:8080`을 열어 두 번째가 죽음

- ① 프로그램마다 포트를 옵션으로 다르게 준다. 이번 실습에서도 서비스가 8001을 쓰면 시험 도구는 8002 이후로 열었고, 8090 충돌로 재실 서비스를 8095로 옮겼다.
- ② 리버스 프록시(nginx) 하나가 8080을 열고 경로별로 나눠 전달한다. 시작 전에 포트가 비었는지 확인하는 것도 도움이 된다.

6-4. 영상이 보드 밖으로 나가는 경로와 막는 법

경로	운영 모드에서 막는 법
웹 뷰어(MJPEG)	끄거나 localhost에만 열기
RTSP 재송출	재송출 없음
스냅샷	저장하지 않음, `/health`의 saved_images = 0으로 확인
녹화	끄기
크롭(재라벨·임베딩용)	메모리에서만 쓰고 파일로 남기지 않기
로그·MQTT	숫자만(인원·상태), 이미지나 base64 없음

과제 6 서비스는 이 방식으로 인원 숫자만 내보낸다. 실습용 `vocab2.py`는 스냅샷을 저장하므로 운영에는 쓰지 않는다.

6-5. `count`와 `raw`

- 제어 로직은 창 평활된 `count`를 쓴다. 한 프레임의 가림·오검출로 값이 튀어도 냉난방·환기가 오락가락하지 않는다.
- `raw`는 지금 순간 값이다. 평활이 늦게 반응하는지 점검하고, 창 크기를 정하고, 카메라 이상을 찾는 데 쓰려고 남긴다.